Rapport de nettoyage SQL – Table appearances

Projet football (données Transfermarkt) – MySQL

1. Vue d'ensemble du projet

L'objectif de cette étape était d'explorer, valider et nettoyer la table `appearances` du jeu de données football, puis de la préparer pour les jointures avec les autres tables (`games` , `players` , `clubs` , `competitions`).

Élément	Valeur
Outil	MySQL (Workbench)
Base source	`raw_football`
Base de destination	`clean_football`
Table source	`appearances`
Table nettoyée	`appearances`
Nombre initial de lignes	1 894 350
Nombre final de lignes	1 894 350

La table d'origine n'a jamais été modifiée, afin de garantir l'intégrité et la traçabilité des données. Tout le nettoyage a été fait dans une table séparée de la base `clean_football` .

2. Exploration initiale

2.1 Aperçu de la table

```
SELECT *
FROM raw_football.appearances
LIMIT 10;
```

2.2 Structure et types de colonnes

```
DESCRIBE raw_football.appearances;
```

La table contient 13 colonnes, toutes stockées en `VARCHAR(255)` à l'import :

Colonne	Contenu
`appearance_id`	Identifiant unique de l'apparition (format `game_id_player_id`)
`game_id`	Identifiant du match
`player_id`	Identifiant du joueur
`player_club_id`	Club du joueur au moment du match
`player_current_club_id`	Club actuel du joueur
`date`	Date du match
`player_name`	Nom du joueur
`competition_id`	Identifiant de la compétition
`yellow_cards`, `red_cards`	Cartons jaunes et rouges
`goals`, `assists`	Buts et passes décisives
`minutes_played`	Minutes jouées

2.3 Comptage des lignes

```
SELECT COUNT(*) AS total_rows
FROM raw_football.appearances;
```

Résultat : 1 894 350 lignes.

3. Évaluation de la qualité des données

3.1 Unicité de `appearance_id`

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT appearance_id) AS unique_appearances
FROM raw_football.appearances;
```

Résultat : 1 894 350 lignes et 1 894 350 identifiants distincts. Aucun doublon : `appearance_id` peut servir de clé primaire.

3.2 Valeurs manquantes (NULL) dans les colonnes d'identification

```
SELECT
    SUM(appearance_id IS NULL) AS missing_appearance_id,
    SUM(game_id IS NULL) AS missing_game_id,
    SUM(player_id IS NULL) AS missing_player_id,
    SUM(date IS NULL) AS missing_date
FROM raw_football.appearances;
```

Résultat : aucune valeur manquante.

3.3 Chaînes vides

Les chaînes vides sont différentes des valeurs `NULL` et doivent être contrôlées séparément.

```
SELECT
    SUM(player_name = '') AS empty_player_names,
    SUM(competition_id = '') AS empty_competition_ids,
    SUM(goals = '') AS empty_goals,
    SUM(assists = '') AS empty_assists,
    SUM(minutes_played = '') AS empty_minutes
FROM raw_football.appearances;
```

Contrôle	Résultat
Noms de joueurs vides	2
`competition_id` vides	0
Buts vides	0
Passes décisives vides	0
Minutes vides	0

3.4 Analyse des noms de joueurs vides

```
SELECT *
FROM raw_football.appearances
WHERE player_name = '';

SELECT *
FROM raw_football.players
WHERE player_id = 380365;
```

Deux enregistrements sont concernés. Le `player_id` correspondant n'existe pas dans la table `players`, donc aucune information complémentaire n'est disponible.

Décision : conserver ces lignes et convertir les noms vides en `NULL` dans la table nettoyée, plutôt que de les supprimer.

3.5 Valeurs numériques invalides

```
SELECT *
FROM raw_football.appearances
WHERE goals NOT REGEXP '^[0-9]+$'
   OR assists NOT REGEXP '^[0-9]+$'
   OR minutes_played NOT REGEXP '^[0-9]+$'
LIMIT 10;
```

Résultat : aucune valeur non numérique.

3.6 Validité des dates

```
SELECT *
FROM raw_football.appearances
WHERE STR_TO_DATE(date, '%Y-%m-%d') IS NULL
LIMIT 10;
```

Résultat : aucune date invalide.

3.7 Valeurs négatives

```
SELECT *
FROM raw_football.appearances
WHERE CAST(goals AS SIGNED) < 0
   OR CAST(assists AS SIGNED) < 0
   OR CAST(minutes_played AS SIGNED) < 0
LIMIT 10;
```

Résultat : aucune valeur négative dans les statistiques.

3.8 Minutes jouées maximales

```
SELECT MAX(CAST(minutes_played AS UNSIGNED)) AS max_minutes
FROM raw_football.appearances;
```

Résultat : 148 minutes. Le match concerné impliquait Javi Hernández. Cette valeur est inhabituelle mais plausible (prolongations et temps additionnel), donc elle est conservée.

3.9 Minutes jouées minimales

```
SELECT MIN(CAST(minutes_played AS UNSIGNED)) AS min_minutes
FROM raw_football.appearances;
```

Résultat : 0 minute. Une valeur de 0 est considérée comme valide (joueur sur la feuille de match sans entrer en jeu) et conservée.

3.10 Amplitudes des statistiques

Les valeurs maximales observées confirment que de petits types entiers suffisent :

Colonne	Minimum	Maximum
`yellow_cards`	0	2
`red_cards`	0	1
`goals`	0	6
`assists`	0	6
`minutes_played`	0	148

3.11 Valeur sentinelle `-1`

La colonne `player_current_club_id` contient la valeur `-1` pour signifier « club actuel inconnu ». Ce n'est pas un identifiant de club réel : elle doit être remplacée par `NULL` pour ne pas fausser les jointures et les calculs.

4. Création de la table nettoyée

4.1 Transformation SQL

```
CREATE TABLE clean_football.appearances AS
SELECT
    CAST(appearance_id AS CHAR(50)) AS appearance_id,
    CAST(game_id AS SIGNED) AS game_id,
    CAST(player_id AS SIGNED) AS player_id,
    CAST(player_club_id AS SIGNED) AS player_club_id,
    CAST(player_current_club_id AS SIGNED) AS player_current_club_id,
    STR_TO_DATE(date, '%Y-%m-%d') AS date,
    NULLIF(TRIM(player_name), '') AS player_name,
    competition_id,
    CAST(yellow_cards AS SIGNED) AS yellow_cards,
    CAST(red_cards AS SIGNED) AS red_cards,
    CAST(goals AS SIGNED) AS goals,
    CAST(assists AS SIGNED) AS assists,
    CAST(minutes_played AS SIGNED) AS minutes_played
FROM raw_football.appearances;
```

4.2 Transformations appliquées

Fonction	Rôle
`CAST()`	Conversion des colonnes numériques stockées en texte vers des types numériques
`STR_TO_DATE()`	Conversion des dates texte vers le type `DATE`
`TRIM()`	Suppression des espaces inutiles dans les noms de joueurs
`NULLIF()`	Conversion des noms vides en `NULL`

5. Finalisation du schéma

Une table créée avec `CREATE TABLE ... AS SELECT` n'a ni clé primaire ni index, et ses types restent trop larges. Un script de finition a donc été appliqué.

```
USE clean_football;

SET SQL_SAFE_UPDATES = 0;
```

```
UPDATE appearances
SET player_current_club_id = NULL
WHERE player_current_club_id = -1;

ALTER TABLE appearances
  MODIFY appearance_id   VARCHAR(20) NOT NULL,
  MODIFY game_id         INT NOT NULL,
  MODIFY player_id       INT NOT NULL,
  MODIFY player_club_id  INT NOT NULL,
  MODIFY competition_id  CHAR(4) NOT NULL,
  MODIFY yellow_cards    TINYINT UNSIGNED,
  MODIFY red_cards       TINYINT UNSIGNED,
  MODIFY goals           TINYINT UNSIGNED,
  MODIFY assists         TINYINT UNSIGNED,
  MODIFY minutes_played  SMALLINT UNSIGNED,
  ADD PRIMARY KEY (appearance_id),
  ADD INDEX idx_game (game_id),
  ADD INDEX idx_player (player_id),
  ADD INDEX idx_club (player_club_id);
```

5.1 Types finaux

Colonne	Type d'origine	Après `CREATE TABLE AS`	Type final
`appearance_id`	VARCHAR(255)	CHAR(50)	VARCHAR(20), clé primaire
`game_id`	VARCHAR(255)	BIGINT	INT, indexé
`player_id`	VARCHAR(255)	BIGINT	INT, indexé
`player_club_id`	VARCHAR(255)	BIGINT	INT, indexé
`player_current_club_id`	VARCHAR(255)	BIGINT	BIGINT, `-1` remplacé par NULL
`date`	VARCHAR(255)	DATE	DATE
`player_name`	VARCHAR(255)	VARCHAR	VARCHAR, vides remplacés par NULL
`competition_id`	VARCHAR(255)	VARCHAR(255)	CHAR(4)
`yellow_cards`, `red_cards`	VARCHAR(255)	BIGINT	TINYINT UNSIGNED
`goals`, `assists`	VARCHAR(255)	BIGINT	TINYINT UNSIGNED

Colonne	Type d'origine	Après `CREATE TABLE AS`	Type final
`minutes_played`	VARCHAR(255)	BIGINT	SMALLINT UNSIGNED

5.2 Pourquoi ces choix

- **Types réduits** : une table plus légère et des requêtes plus rapides sur 1,9 million de lignes.
- **Clé primaire sur `appearance_id`** : garantit l'unicité et accélère la recherche d'une ligne.
- **Index sur `game_id`, `player_id` et `player_club_id`** : ces colonnes servent aux jointures avec `games`, `players` et `clubs`.
- **`NOT NULL`** sur les identifiants : une apparition doit toujours être rattachée à un match, un joueur et un club.
- **`NULL` à la place de `-1`** : un identifiant inconnu ne doit pas ressembler à un vrai club.

6. Validation finale

6.1 Structure de la table

```
DESCRIBE clean_football.appearances;
```

À vérifier : `appearance_id` apparaît avec `PRI` dans la colonne Key, et les types correspondent au tableau de la section 5.1.

6.2 Nombre de lignes et valeurs inconnues

```
SELECT COUNT(*) AS total_rows,
       SUM(player_current_club_id IS NULL) AS clubs_inconnus,
       SUM(player_current_club_id = -1)    AS encore_moins_un
FROM clean_football.appearances;
```

Résultat obtenu :

total_rows	clubs_inconnus	encore_moins_un
1 894 350	2	0

La table contient 1 894 350 lignes. Seules 2 lignes avaient la valeur sentinelle `-1` ; elles sont maintenant `NULL`, et il n'en reste aucune. Le premier `UPDATE` avait été bloqué par le mode sécurité de Workbench (erreur 1175) : il a été relancé avec `SET SQL_SAFE_UPDATES = 0;` exécuté dans le même lot.

6.3 Comparaison source et table nettoyée

```
SELECT
    (SELECT COUNT(*) FROM raw_football.appearances)   AS raw_rows,
    (SELECT COUNT(*) FROM clean_football.appearances) AS clean_rows;
```

Base	Nombre de lignes
`raw_football`	1 894 350
`clean_football`	1 894 350

Aucune ligne n'a été supprimée pendant le nettoyage.

7. Points d'attention pour la suite

Plusieurs identifiants de la table `appearances` n'ont pas de correspondance dans les autres tables du jeu de données :

Relation	Statut
`appearances.game_id` → `games.game_id`	Aucune ligne orpheline : clé étrangère possible
`appearances.player_id` → `players.player_id`	2 lignes orphelines (les deux apparitions sans nom de joueur)
`appearances.player_club_id` → `clubs.club_id`	Plus de 11 000 lignes orphelines : la table `clubs` ne contient que 796 clubs

Recommandations :

- Ajouter la clé étrangère vers `games` seulement après avoir nettoyé la table `games`.
- Ne pas poser de clé étrangère vers `clubs` : conserver de simples index et utiliser `LEFT JOIN` dans les analyses.
- Traiter les 2 lignes orphelines avant toute clé étrangère vers `players`.
- Utiliser `LEFT JOIN` plutôt que `INNER JOIN` quand on relie `appearances` aux tables `clubs` et `players`, pour ne pas perdre de lignes.

8. Conclusion

La table `appearances` a été explorée, validée et transformée avec succès.

Les opérations principales :

1. Compréhension de la structure et du contenu des données.
2. Vérification des identifiants uniques et des valeurs manquantes.
3. Détection des noms de joueurs vides (2 cas).
4. Validation des valeurs numériques, des dates et des plages de valeurs.
5. Analyse des valeurs inhabituelles (148 minutes) et des valeurs sentinelles (`-1`).
6. Conversion des colonnes vers des types adaptés.
7. Création d'une table séparée, puis ajout de la clé primaire et des index.
8. Vérification du nombre de lignes et de la structure finale.

Les données d'origine sont intactes dans `raw_football` . La table `clean_football.appearances` contient 1 894 350 lignes, avec des types optimisés, une clé primaire et des index. Elle est prête pour les analyses SQL et les jointures avec les autres tables du projet.

Prochaine étape : nettoyage de la table `clubs` .

9. Table clubs

La table `clubs` (796 lignes) a été nettoyée selon la même démarche que `appearances` : table brute intacte dans `raw_football` , table propre créée dans `clean_football` .

9.1 Contrôles de qualité sur la table brute

```
SELECT COUNT(*)                                AS total,
       COUNT(DISTINCT club_id)                 AS ids_uniques,
       SUM(club_id NOT REGEXP '^[0-9]+$')      AS id_non_numerique,
       SUM(total_market_value = '')            AS valeur_marche_vide,
       SUM(coach_name = '')                    AS coach_vide,
       SUM(average_age = '')                   AS age_vide,
       SUM(average_age = '0')                  AS age_zero,
       SUM(foreigners_percentage = '')         AS pct_etrangers_vide,
       SUM(squad_size = '0')                   AS squad_zero,
       SUM(stadium_seats = '0')                AS stade_zero
FROM raw_football.clubs;
```

Contrôle	Résultat	Décision
Lignes / `club_id` uniques	796 / 796	Aucun doublon : `club_id` devient clé primaire
`club_id` non numériques	0	Rien à corriger
`total_market_value` vide	796	Colonne vide partout : supprimée
`coach_name` vide	393	Converti en `NULL`
`average_age` vide	38	Converti en `NULL`
`foreigners_percentage` vide	60	Converti en `NULL`
`squad_size = 0`	42	Effectif inconnu : converti en `NULL`
`stadium_seats = 0`	1	Converti en `NULL`

9.2 Transformations appliquées

- Colonnes `total_market_value`, `filename` et `url` non conservées (vide ou inutile pour l'analyse).
- Cellules vides et valeurs `0` signifiant « inconnu » remplacées par `NULL` (`NULLIF`).
- Conversion de `net_transfer_record` (texte comme `+€2.69m`, `€-25.00m`, `+€40k`) en un nombre entier en euros, dans la colonne `net_transfer_eur`.
- Types adaptés : `INT` pour `club_id`, `SMALLINT UNSIGNED` pour les effectifs, `CHAR(4)` pour `domestic_competition_id`, `BIGINT` pour `net_transfer_eur`.
- Clé primaire sur `club_id` et index sur `domestic_competition_id`.

9.3 Validation

Contrôle	Résultat
Lignes dans la table propre	796
`coach_name` NULL	393
`average_age` NULL	42
`squad_size` NULL	42
`stadium_seats` NULL	1
`net_transfer_eur` NULL	0

La table propre contient autant de lignes que la table brute (796). Le nombre de `NULL` sur `average_age` (42) dépasse les 38 cellules vides détectées au contrôle : le script convertit aussi les valeurs à zéro en `NULL`, ce qui correspond aux 42 clubs sans effectif.

Le top 5 de `net_transfer_eur` confirme que la conversion est correcte (montants en millions d'euros) :

Club	net_transfer_eur
Nottingham Forest	144 000 000
West Ham United	97 000 000
Newcastle United	93 000 000
Southampton FC	86 000 000
Stade Reims	81 000 000

Un montant positif correspond à un solde de transferts excédentaire (plus de ventes que d'achats), un montant négatif à un solde déficitaire.

10. Table players

La table `players` (50 149 joueurs) a été nettoyée avec la même démarche : table brute intacte dans `raw_football`, table propre créée dans `clean_football`. Les 26 colonnes étaient toutes en `VARCHAR` à l'import.

10.1 Contrôles de qualité sur la table brute

```
SELECT COUNT(*)                                      AS total,
       COUNT(DISTINCT player_id)                     AS ids_uniques,
       SUM(player_id NOT REGEXP '^[0-9]+$')          AS id_non_numerique,
       SUM(first_name = '')                          AS prenom_vide,
       SUM(date_of_birth = '')                       AS naissance_vide,
       SUM(height_in_cm = '')                        AS taille_vide,
       SUM(CAST(NULLIF(height_in_cm,'') AS DECIMAL(5,1)) < 140) AS
taille_sous_140,
       SUM(market_value_in_eur = '')                 AS valeur_vide,
       SUM(contract_expiration_date = '')            AS contrat_vide
FROM raw_football.players;
```

Contrôle	Résultat	Décision
Lignes / `player_id` uniques	50 149 / 50 149	Aucun doublon : `player_id` devient clé primaire
`player_id` non numériques	0	Rien à corriger
`first_name` vide	3 164	Joueurs connus sous un seul nom : `NULL`
`date_of_birth` vide	49	`NULL`
`height_in_cm` vide	4 365	`NULL`
Tailles sous 140 cm	13	Erreurs de saisie : `NULL`
`market_value_in_eur` vide	8 621	`NULL`
`contract_expiration_date` vide	18 569	`NULL`

10.2 Transformations appliquées

- Colonnes `image_url` et `url` non conservées (inutiles pour l'analyse) : 24 colonnes sur 26 gardées.
- Cellules vides remplacées par `NULL` (`NULLIF`) et espaces de fin supprimés (`TRIM`), par exemple `Opole ` dans `city_of_birth`.
- `date_of_birth` et `contract_expiration_date` converties en vrai type `DATE` (l'heure inutile `00:00:00` est supprimée).
- Les 13 tailles inférieures à 140 cm remplacées par `NULL`.
- Types adaptés : `INT` pour les identifiants, `SMALLINT UNSIGNED` pour la taille et les sélections, `BIGINT UNSIGNED` pour les valeurs marchandes, `CHAR(4)` pour `current_club_domestic_competition_id`.
- Clé primaire sur `player_id`, index sur `current_club_id` et `current_club_domestic_competition_id`.
- Les anciens noms de pays (`East Germany (GDR)`, `CSSR`) sont conservés : ce sont les pays de naissance historiques.

10.3 Validation

Contrôle	Résultat	Attendu
Lignes dans la table propre	50 149	50 149

Contrôle	Résultat	Attendu
`first_name` NULL	3 164	3 164
`date_of_birth` NULL	49	49
`height_in_cm` NULL	4 378	4 378 (4 365 + 13)
`market_value_in_eur` NULL	8 621	8 621
`contract_expiration_date` NULL	18 569	18 569
Taille minimale	154 cm	140 cm ou plus
Taille maximale	210 cm	210 cm

Tous les résultats correspondent aux valeurs attendues et aucune ligne n'a été supprimée.

Points d'attention pour les analyses :

- La table contient aussi des joueurs retraités ou anciens : leur club et leur valeur marchande sont ceux de leur dernière saison connue (`last_season`). Pour les joueurs actifs, filtrer sur `last_season` .
- Les valeurs marchandes manquantes (8 621) sont des `NULL` : ne pas les traiter comme zéro dans les moyennes.

11. Table games

La table `games` (88 958 matchs) a été nettoyée avec la même démarche : table brute intacte dans `raw_football` , table propre créée dans `clean_football` . Les 23 colonnes étaient toutes en `VARCHAR` à l'import.

11.1 Contrôles de qualité sur la table brute

```
SELECT COUNT(*)                                        AS total,
       COUNT(DISTINCT game_id)                         AS ids_uniques,
       SUM(game_id NOT REGEXP '^[0-9]+$')              AS id_non_numerique,
       SUM(STR_TO_DATE(date, '%Y-%m-%d') IS NULL)      AS date_invalide,
       SUM(referee = '')                               AS arbitre_vide,
       SUM(stadium = '')                               AS stade_vide,
       SUM(attendance = '')                            AS affluence_vide,
       SUM(home_club_position = '')                    AS position_dom_vide,
       SUM(home_club_formation = '')                   AS formation_dom_vide,
       SUM(competition_type = '')                      AS type_comp_vide
FROM raw_football.games;
```

Contrôle	Résultat	Décision
Lignes / `game_id` uniques	88 958 / 88 958	Aucun doublon : `game_id` devient clé primaire
`game_id` non numériques	0	Rien à corriger
Dates invalides	0	Conversion directe en `DATE`
`referee` vide	678	`NULL`
`stadium` vide	242	`NULL`
`attendance` vide	10 803	`NULL`
`home_club_position` vide	25 377	`NULL` (matchs de coupe et de sélections, sans classement)
`home_club_formation` vide	8 200	`NULL`
`competition_type` vide	1 214	`NULL`

11.2 Transformations appliquées

- Colonne `url` non conservée (inutile pour l'analyse) : 22 colonnes sur 23 gardées.
- Cellules vides remplacées par `NULL` (`NULLIF`) et espaces inutiles supprimés (`TRIM`).
- `date` convertie en vrai type `DATE` (`STR_TO_DATE`).
- Types adaptés : `INT` pour les identifiants de match et de clubs, `TINYINT UNSIGNED` pour les buts et les positions, `INT UNSIGNED` pour l'affluence, `CHAR(4)` pour `competition_id` .
- Clé primaire sur `game_id` , index sur `competition_id` , `home_club_id` , `away_club_id` , `date` et `season` .
- Aucune clé étrangère vers `competitions` ni `clubs` : de nombreux matchs référencent des compétitions ou des clubs absents de ces tables, ce qui bloquerait la création.

11.3 Validation

Contrôle	Résultat	Attendu
Lignes dans la table propre	88 958	88 958
`referee` NULL	678	678

Contrôle	Résultat	Attendu
`stadium` NULL	242	242
`attendance` NULL	10 803	10 803
`home_club_position` NULL	25 377	25 377
`home_club_formation` NULL	8 200	8 200
`competition_type` NULL	1 214	1 214
Date minimale	2006-06-09	2006-06-09
Date maximale	2026-07-06	2026-07-06

Tous les résultats correspondent aux valeurs attendues et aucune ligne n'a été supprimée. Le `ALTER TABLE` s'est exécuté sans erreur : aucune valeur ne dépassait la taille prévue pour sa colonne.

Points d'attention pour les analyses :

- Les 1 214 matchs sans `competition_type` sont ceux dont la `competition_id` n'existe pas dans la table `competitions` : utiliser un `LEFT JOIN` pour ne pas les perdre.
- Beaucoup de matchs référencent des clubs absents de la table `clubs` (qui ne contient que 796 clubs) : même précaution avec `LEFT JOIN`.
- Les noms de formation (`home_club_formation`, `away_club_formation`) sont conservés tels quels : certaines valeurs sont longues (jusqu'à 33 caractères) et pourraient être normalisées plus tard si une analyse en a besoin.
- Les positions au classement sont absentes pour les compétitions sans classement (coupes, sélections) : ne pas les traiter comme zéro.

12. Table transfers

La table `transfers` (175 165 transferts) a été nettoyée avec la même démarche : table brute intacte dans `raw_football`, table propre créée dans `clean_football`. Les colonnes étaient toutes en `VARCHAR` à l'import.

12.1 Contrôles de qualité sur la table brute

```
SELECT COUNT(*)                                          AS total,
       SUM(STR_TO_DATE(transfer_date, '%Y-%m-%d') IS NULL) AS date_invalide,
       SUM(transfer_date > CURDATE())                    AS date_future,
       SUM(transfer_fee = '')                            AS frais_vide,
       SUM(transfer_fee = '0' OR transfer_fee = '0.0')   AS frais_zero,
       SUM(market_value_in_eur = '')                     AS valeur_vide,
       SUM(player_name = '')                             AS nom_vide,
       SUM(from_club_id = to_club_id)                    AS meme_club
FROM raw_football.transfers;
```

Contrôle	Résultat	Décision
Lignes	175 165	
Dates invalides	0	Conversion directe en `DATE`
Dates dans le futur	475	Conservées, signalées par la colonne `is_future`
`transfer_fee` vide	61 526	`NULL` (frais inconnus)
`transfer_fee` à `'0'` ou `'0.0'` (texte)	0	Le format réel est différent : contrôle refait en comparant des nombres (section 12.3)
`market_value_in_eur` vide	68 160	`NULL`
`player_name` vide	0	Rien à corriger
Même club au départ et à l'arrivée	8	Conservés (cas rare mais possible)

12.2 Transformations appliquées

- `transfer_date` convertie en vrai type `DATE`.
- Cellules vides remplacées par `NULL` (`NULLIF`) et espaces inutiles supprimés (`TRIM`).
- Montants (`transfer_fee`, `market_value_in_eur`) convertis en entiers en euros via `DECIMAL`, qui accepte les formats `0`, `0.0` et `0.00`.
- Ajout d'une clé primaire `transfer_id` (auto-incrément), car la table d'origine n'a pas d'identifiant unique par transfert.
- Ajout d'une colonne `is_future` (1 si la date est postérieure à la date du jour, 0 sinon) pour pouvoir exclure facilement les 475 transferts futurs avec `WHERE is_future = 0`.
- Index sur `player_id`, `from_club_id`, `to_club_id` et `transfer_date`.

- Aucune clé étrangère vers `clubs` : de nombreux clubs d'origine ou de destination sont absents de cette table.

12.3 Validation

Contrôle	Résultat	Attendu
Lignes dans la table propre	175 165	175 165
`transfer_fee` NULL	61 526	61 526
`market_value_in_eur` NULL	68 160	68 160
Transferts futurs (`is_future = 1`)	475	475
Date minimale	1993-07-01	environ 1993-07-01
Date maximale	2030-06-30	environ 2030-06-30
Frais maximal	222 000 000	environ 222 000 000
Transferts gratuits (`transfer_fee = 0`)	96 085	96 085 (table brute)

Le nombre de transferts gratuits est identique dans la table brute (comparé en valeur numérique) et dans la table propre : aucun transfert à 0 € n'a été perdu ni transformé en `NULL` . Aucune ligne n'a été supprimée.

Points d'attention pour les analyses :

- Un `transfer_fee` à 0 signifie un transfert gratuit (prêt, fin de contrat, jeune), alors qu'un `NULL` signifie des frais inconnus : les deux ne doivent pas être confondus. Dans une moyenne des frais, exclure les `NULL` et décider explicitement si les 96 085 transferts gratuits comptent.
- Les 475 transferts futurs (jusqu'en 2030) sont probablement des fins de contrat ou des mouvements annoncés : filtrer avec `WHERE is_future = 0` pour ne garder que les transferts déjà réalisés.
- La colonne `is_future` a été calculée à la date de création de la table : elle n'est pas mise à jour automatiquement.
- Utiliser `LEFT JOIN` pour relier `transfers` à `clubs` , car beaucoup de clubs sont absents de la table `clubs` .

13. Table player_valuations

La table `player_valuations` (656 301 valorisations) a été nettoyée avec la même démarche : table brute intacte dans `raw_football`, table propre créée dans `clean_football`. Les 6 colonnes étaient toutes en `VARCHAR` à l'import.

13.1 Contrôles de qualité sur la table brute

```
SELECT COUNT(*)                                          AS total,
       SUM(STR_TO_DATE(date, '%Y-%m-%d') IS NULL)        AS date_invalide,
       SUM(market_value_in_eur = '')                     AS valeur_vide,
       SUM(CAST(NULLIF(market_value_in_eur,'') AS DECIMAL(15,2)) = 0) AS
valeur_zero,
       SUM(current_club_name = 'Unknown')                AS club_unknown,
       SUM(current_club_name = '')                       AS club_vide,
       SUM(player_club_domestic_competition_id = '')     AS competition_vide
FROM raw_football.player_valuations;

SELECT COUNT(*) AS lignes_en_double
FROM (
    SELECT player_id, date
    FROM raw_football.player_valuations
    GROUP BY player_id, date
    HAVING COUNT(*) > 1
) AS d;
```

Contrôle	Résultat	Décision
Lignes	656 301	
Dates invalides	0	Conversion directe en `DATE`
`market_value_in_eur` vide	0	Rien à corriger
`market_value_in_eur` à 0	1	Valeur suspecte : `NULL`
`current_club_name = 'Unknown'`	3 199	Valeur « inconnu » écrite en texte : `NULL`
`current_club_name` vide	0	Rien à corriger
`player_club_domestic_competition_id` vide	92 973	`NULL`
Doublons sur `(player_id, date)`	0	Ce couple devient la clé primaire

13.2 Transformations appliquées

- `date` convertie en vrai type `DATE`.
- `current_club_name` : les valeurs `Unknown` et les cellules vides remplacées par `NULL`.
- `market_value_in_eur` : conversion en entier, et la seule valeur à 0 remplacée par `NULL` (valeur inconnue plutôt que valeur réelle à zéro).
- `player_club_domestic_competition_id` : cellules vides remplacées par `NULL`.
- Types adaptés : `INT` pour les identifiants, `INT UNSIGNED` pour la valeur marchande (maximum observé : 200 000 000), `CHAR(4)` pour l'identifiant de compétition, `VARCHAR(60)` pour le nom du club.
- Clé primaire composite sur `(player_id, date)` : la table d'origine n'a pas d'identifiant propre, et ce couple est unique.
- Index sur `current_club_id` et `date`.
- Aucune clé étrangère vers `players` ou `clubs`.

13.3 Validation

Contrôle	Résultat	Attendu
Lignes dans la table propre	656 301	656 301
Joueurs distincts	41 528	41 528
`market_value_in_eur` NULL	1	1
`current_club_name` NULL	3 199	3 199
`player_club_domestic_competition_id` NULL	92 973	92 973
Date minimale	2000-01-20	2000-01-20
Date maximale	2026-06-12	2026-06-12
Valeur maximale	200 000 000	200 000 000

Tous les résultats correspondent aux valeurs attendues et aucune ligne n'a été supprimée.

Points d'attention pour les analyses :

- Chaque joueur a plusieurs valorisations dans le temps (656 301 lignes pour 41 528 joueurs) : pour obtenir la valeur actuelle d'un joueur, prendre la ligne avec la date la plus récente (`MAX(date)`), ou utiliser la colonne `market_value_in_eur` de la table `players`.
- Les valeurs `NULL` de `market_value_in_eur` et de `current_club_name` ne doivent pas être traitées comme zéro ou comme un club réel.

- Utiliser `LEFT JOIN` pour relier cette table à `players` et `clubs`, car certains identifiants n'existent pas dans ces tables.

14. Table competitions

La table `competitions` (65 compétitions) a été nettoyée avec la même démarche : table brute intacte dans `raw_football`, table propre créée dans `clean_football`. Les 11 colonnes étaient toutes en `VARCHAR` à l'import.

14.1 Contrôles de qualité sur la table brute

```
SELECT COUNT(*)                                    AS total,
       COUNT(DISTINCT competition_id)              AS ids_uniques,
       SUM(competition_id IS NULL)                 AS id_null,
       SUM(country_id = '-1')                      AS pays_moins_un,
       SUM(country_name = '')                      AS pays_nom_vide,
       SUM(domestic_league_code = '')              AS ligue_vide,
       SUM(total_clubs = '')                       AS clubs_vide,
       SUM(sub_type = '')                          AS sous_type_vide,
       SUM(type = '')                              AS type_vide
FROM raw_football.competitions;
```

Contrôle	Résultat	Décision
Lignes / `competition_id` uniques	65 / 65	Aucun doublon : `competition_id` devient clé primaire
`competition_id` NULL	0	Identifiants complets
`country_id = -1`	12	Valeur « inconnu » des compétitions internationales : `NULL`
`country_name` vide	12	`NULL`
`domestic_league_code` vide	12	`NULL`
`total_clubs` vide	15	`NULL`
`sub_type` et `type` vides	0 / 0	Rien à corriger

14.2 Transformations appliquées

- Colonne `url` non conservée (inutile pour l'analyse) : 10 colonnes sur 11 gardées.
- `country_id = -1` remplacé par `NULL` et cellules vides remplacées par `NULL` (`NULLIF`).

- Espaces inutiles supprimés (`TRIM`).
- `name` et `competition_code` conservés tels quels (minuscules et tirets, par exemple `afc-asian-cup`) pour ne pas casser les jointures.
- Types adaptés : `CHAR(4)` pour `competition_id` et `domestic_league_code` , `INT` pour `country_id` , `TINYINT UNSIGNED` pour `total_clubs` .
- Clé primaire sur `competition_id` et index sur `country_id` .

14.3 Validation

Contrôle	Résultat	Attendu
Lignes dans la table propre	65	65
Types de compétition distincts	5	5
`country_id` NULL	12	12
`country_name` NULL	12	12
`domestic_league_code` NULL	12	12
`total_clubs` NULL	15	15
Matchs de `games` sans compétition connue	1 214	environ 1 214

Tous les résultats correspondent aux valeurs attendues et aucune ligne n'a été supprimée.

Points d'attention pour les analyses :

- Les 1 214 matchs de `games` dont la `competition_id` est absente de cette table sont exactement ceux dont `competition_type` est `NULL` dans `games` . Utiliser un `LEFT JOIN` pour ne pas les perdre.
- Les 12 compétitions sans pays sont des compétitions internationales ou de sélections (Coupe d'Asie, Coupe d'Afrique…) : un `NULL` signifie ici « pas de pays unique », pas une erreur.

15. Conclusion générale

Les sept tables du jeu de données football (données Transfermarkt) ont été explorées, contrôlées, nettoyées et validées dans MySQL. Pour chacune, la table brute est restée intacte dans `raw_football` et la table propre a été créée dans `clean_football` , avec une vérification systématique du nombre de lignes. Aucune ligne n'a été supprimée au cours du nettoyage.

15.1 Récapitulatif des tables

Table	Lignes	Clé primaire	Principaux traitements
`appearances`	1 894 350	`appearance_id`	Types numériques et `DATE`, noms vides en `NULL`, `-1` remplacé par `NULL` (2 lignes), 3 index
`clubs`	796	`club_id`	Colonne vide et colonnes inutiles retirées, `net_transfer_record` converti en euros, `0` « inconnu » en `NULL`
`players`	50 149	`player_id`	Dates sans heure, 13 tailles aberrantes en `NULL`, cellules vides en `NULL`, colonnes `image_url` et `url` retirées
`games`	88 958	`game_id`	Dates en `DATE`, cellules vides en `NULL`, 5 index, colonne `url` retirée
`transfers`	175 165	`transfer_id` (créé)	Dates en `DATE`, colonne `is_future` (475 transferts futurs), frais gratuits (0) distingués des frais inconnus (`NULL`)
`player_valuations`	656 301	`(player_id, date)`	`Unknown` et valeur à 0 en `NULL`, dates en `DATE`
`competitions`	65	`competition_id`	`-1` et cellules vides en `NULL`, colonne `url` retirée

15.2 Bonnes pratiques appliquées

- Séparer les données brutes (`raw_football`) des données nettoyées (`clean_football`).
- Contrôler les identifiants (unicité, valeurs manquantes), les chaînes vides, les formats numériques et les dates avant toute conversion.
- Distinguer `NULL` (information absente) de `0` (valeur réelle) et remplacer les valeurs sentinelles (`-1`, `Unknown`) par `NULL`.
- Conserver les valeurs inhabituelles mais plausibles (par exemple 148 minutes jouées) au lieu de les supprimer.
- Choisir des types adaptés à la taille réelle des données et ajouter clés primaires et index sur les colonnes de jointure.

- Vérifier chaque table avec une requête de validation avant de passer à la suivante.

15.3 Points d'attention pour les jointures

Les tables ne sont pas parfaitement cohérentes entre elles : plusieurs identifiants référencés dans une table n'existent pas dans la table liée.

Relation	Constat
`games.competition_id` → `competitions`	1 214 matchs sans compétition connue (vérifié)
`appearances.player_club_id`, `games.home_club_id` et `away_club_id`, `transfers.from_club_id` et `to_club_id` → `clubs`	Nombreux clubs absents : la table `clubs` ne contient que 796 clubs
`appearances.player_id` → `players`	2 apparitions sans joueur correspondant

Recommandations :

- Utiliser `LEFT JOIN` plutôt que `INNER JOIN` pour relier les tables entre elles, afin de ne pas perdre de lignes.
- Ne pas créer de clés étrangères vers `clubs` ni vers `competitions` : elles seraient refusées par MySQL.
- Les clés étrangères `appearances.game_id` → `games`, `transfers.player_id` → `players` et `player_valuations.player_id` → `players` sont possibles d'après l'analyse initiale des fichiers, mais doivent être testées dans MySQL avant d'être créées.
- Filtrer les transferts futurs avec `WHERE is_future = 0` et ne pas traiter les `NULL` comme des zéros dans les moyennes.

15.4 Suite possible du projet

1. Vérifier la cohérence entre les tables (clés étrangères possibles, lignes orphelines).
2. Créer des vues ou des tables d'analyse (par exemple buts et passes par joueur et par saison, bilan des transferts par club, évolution de la valeur marchande d'un joueur).
3. Construire un tableau de bord ou un modèle prédictif (par exemple la valeur marchande d'un joueur) à partir des tables nettoyées.

État final : nettoyage des 7 tables terminé, les données sont prêtes pour l'analyse SQL.